

EE675

Assignment 1

1(a). Simulator Function :

It models the dynamics of the predator-prey environment. At each time step, the predator takes an action, while the prey moves randomly among its valid neighboring cells or stays in place. Boundary conditions are handled by preventing movement outside the grid.

If the predator reaches the same cell as the prey, a reward of +1 is given. The prey is then respawned at a random location different from the predator's position. Otherwise, the reward is 0. This function effectively defines the transition dynamics of the Markov Decision Process (MDP).

1(b). Kernel Function :

It represents the probability of moving from one state to another under a given action. Due to the stochastic movement of the prey, transitions are probabilistic.

In practice, instead of computing the full kernel (which is computationally expensive due to $O(N^4)$ states), transitions are approximated using sampling. Multiple simulations are performed for each state-action pair to estimate transition probabilities.

1(c). Reward Function :

It assigns a value to each state-action pair. The predator receives a reward of +1 when it captures the prey, and 0 otherwise.

This function is deterministic given the current state and action, and directly reflects the objective of the predator in the environment.

1(d). Sample_policy :

It is a stochastic policy designed to balance exploitation and exploration. With probability 0.5, the predator chooses the action that minimizes the Manhattan distance to the prey. The remaining probability is distributed uniformly among other valid actions.

This ensures that the predator generally moves toward the prey while still exploring alternative actions.

1(e). Policy Induced Kernel Function :

It represents the transition probabilities when actions are selected according to a given policy.

For stochastic policies, the induced kernel is computed as a weighted combination of transition probabilities over all actions, where weights correspond to policy probabilities. This converts the MDP into a Markov chain under the given policy.

1(f). Policy Induced Reward Function :

It is the expected reward under a given policy. It is computed as the weighted sum of rewards over all actions, using the policy probabilities.

This reduces the reward function from state-action form to a state-based expectation.

1(g). State Value Evaluation Function :

It represents the expected discounted return starting from each state under a given policy.

Instead of using matrix inversion, an iterative approach (Bellman expectation update) is used. The value of each state is updated based on sampled next states and rewards.

To handle the large state space, a sampling-based approximation is used. For transitions to states outside the sampled subset, the average value of known states is used to maintain stability.

1(h). Q-Value Function:

It represents the expected return for each state-action pair.

It is computed using the Bellman equation:

- First, Q-values are updated using expected rewards and future values.
- Then, the state value function is updated using the policy.

This process is repeated iteratively until convergence.

1(g). State Value Evaluation Function

The state value function represents the expected discounted return starting from a given state under a fixed policy. It is defined using the Bellman expectation equation

Instead of using the matrix inversion method, which is computationally expensive for large state spaces, an iterative approach is used. In each iteration, the value of a state is updated based on the expected reward and the discounted value of the next state.

Due to the large size of the state space ($O(N^4)$), the exact transition probabilities are not explicitly computed. Instead, a sampling-based approach is used. For each state-action pair, multiple simulations are performed using the simulator to estimate the expected next state and reward.

The value update is computed as the weighted sum over actions using the given policy. For transitions that lead to states within the sampled subset, their value is directly used. For transitions to states outside the sampled subset, an approximate value (such as the average value of known states) is used to maintain stability.

The process is repeated iteratively until convergence is achieved or a maximum number of iterations is reached. This approach significantly reduces computational complexity while still providing a reasonable estimate of the value function.

Computational Considerations :

The state space grows as $O(N^4)$, making exact computation infeasible for large N . To address this:

- A subset of states is sampled (approximately 1500 states)
- Transition probabilities are estimated via simulation
- Iterative methods are used instead of matrix inversion

These approximations allow efficient computation while preserving the overall trends in value and runtime.

PLOT 1: State Value vs N

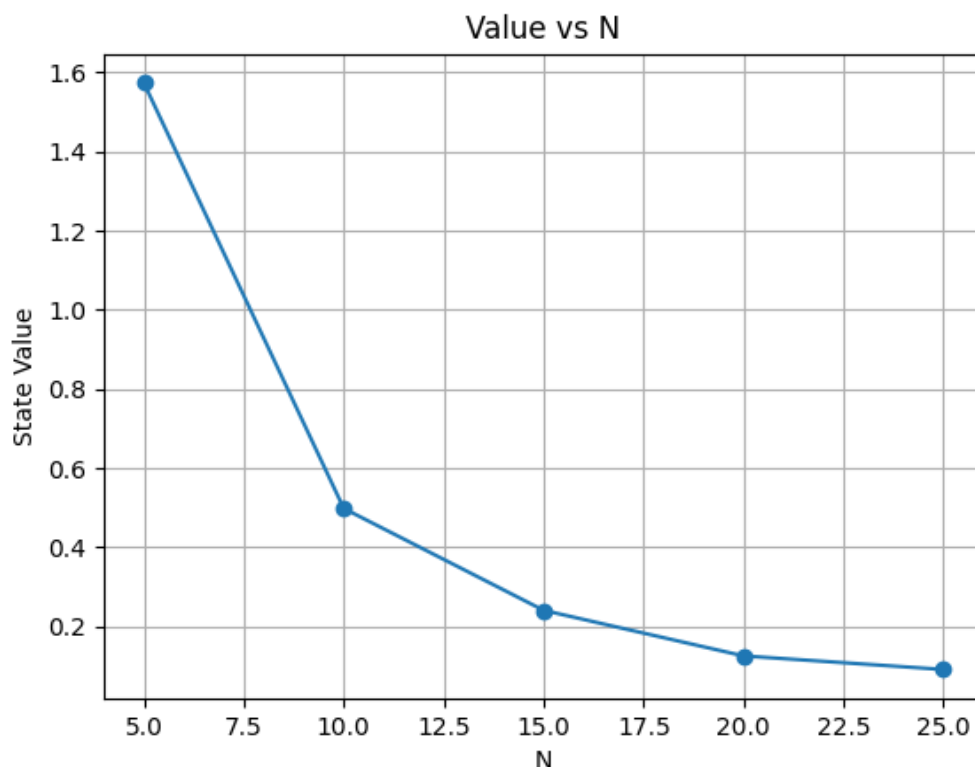


Figure 1: State Value vs N showing decrease in value with increasing grid size.

Plot 2: Runtime vs N

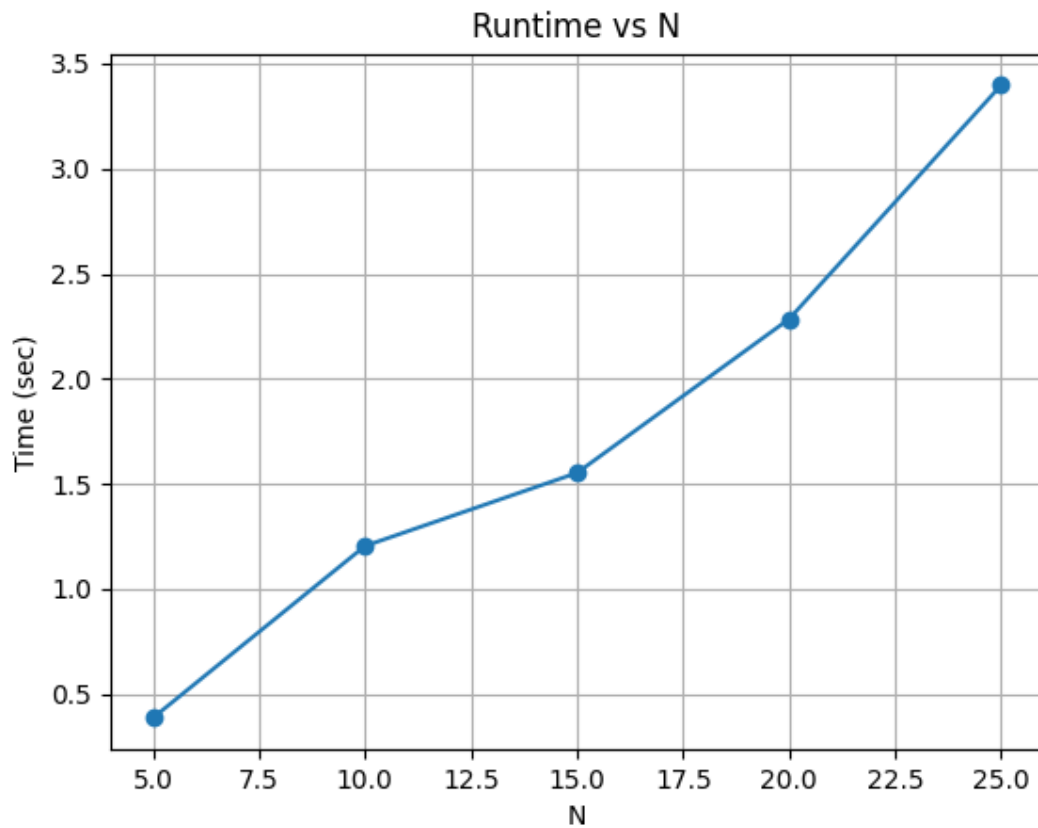


Figure 2: Runtime vs N showing increasing computational cost with larger grids.